

Automatic Detection of Dolphin Whistles and Clicks Based on an Entropy Approach

A Reproduction Study

Reproducing: Siddagangaiah, S., Chen, C.-F., Hu, W.-C., Akamatsu, T., McElligott, M., Lammers, M.O., Pieretti, N.
(2020).

“Automatic detection of dolphin whistles and clicks based on entropy approach.”

Ecological Indicators, 117, 106559. <https://doi.org/10.1016/j.ecolind.2020.106559>

Code repository under review:

github.com/nikhatragnha/Automatic-detection-of-dolphin-whistles-and-clicks-based-on-entropy-approach

Executive Summary

This report documents the outcome of an internship project whose goal was to independently reproduce, from scratch, the entropy-based dolphin whistle and click detector described in Siddagangaiah et al. (2020, Ecological Indicators). The project produced a working, well-tested codebase that implements both entropy metrics used in the paper — permutation entropy (H) and sample entropy (SE) — directly from their mathematical definitions, without relying on any third-party entropy library.

The honest headline finding is that this is a partial, not a full, reproduction. The algorithms themselves were successfully implemented, unit-tested, and validated on the eight short audio clips that the original authors deposited on Mendeley Data. However, the paper's headline quantitative results — the confusion-matrix accuracies (96.6% for whistles, 97.8% for clicks) and the Pearson correlations ($r > 0.95$, $r = 0.89$, $r = 0.97$) between the entropy metrics and manually annotated vocalization activity — cannot be regenerated with the publicly available data. Those figures were computed by the original authors on long-duration field recordings with manual on/off annotations that were never publicly released. This gap is documented transparently in the repository rather than papered over, which is itself one of the more valuable outcomes of the internship.

The sections below walk through what the repository contains, how the two entropy algorithms were implemented, what could be verified against the paper, what remains blocked and why, and what a future extension of this work would need in order to close the gap.

1. Background and Objective

Passive acoustic monitoring (PAM) is widely used to study cetacean populations non-invasively, but manually screening long hydrophone recordings for whistles and clicks is slow and expensive. Siddagangaiah et al. (2020) proposed using two information-theoretic measures — permutation entropy and sample entropy — as fast, unsupervised, training-free detectors that need no labelled vocalization examples. The paper reports that these metrics correlate strongly with independently annotated dolphin acoustic activity and remain robust to common marine noise sources such as snapping shrimp choruses, vessel engine noise, and overlapping whistle/click events.

The internship objective was to reproduce this pipeline end-to-end: implement both entropy estimators from the equations in the paper, apply the documented preprocessing, run the detector on the same or equivalent data, and compare the resulting accuracy and correlation figures against the paper's Table 1 and Figures 1, 2, 6 and 7. Where a step could not be validated against real data, the objective shifted to building and unit-testing the code path so it is ready to run the moment suitable data becomes available.

2. Repository Overview

The codebase is organised as a small, readable research repository rather than a single notebook. Its structure mirrors a typical reproducibility-audit layout: algorithms and orchestration are kept separate, every run is testable, and every unresolved question about the paper's methodology is written down instead of silently assumed.

Folder / File	Contents	Purpose
docs/	paper_parameters.md, reproduction_audit.md, author_correspondence.md, final_report.md	Documentation: parameter ledger, honest status audit, correspondence draft, generated final report
data/raw/	8 WAV files (not committed — SOURCE.md points to Mendeley Data DOI 10.17632/zv6ygrptr4.1)	Original short validation clips released by the paper's authors
data/processed/	Filtered audio, entropy time series (generated)	Intermediate artifacts produced by the pipeline
src/	permutation_entropy.py, sample_entropy.py, and supporting modules	From-scratch algorithm implementations
notebooks/	02 through 10, run in numeric order	Orchestration scripts that import from src/ and produce results
tests/	pytest suite	Unit tests for the entropy algorithms and pipeline stages
results/	figures/, tables/, metrics/ (generated)	Output artifacts from running the notebooks

Two details are worth calling out because they reflect good research-engineering practice rather than a cosmetic choice:

- No existing entropy/complexity library — statcomp, EntropyHub, nolds, antropy, pyEntropy — was used for the core computation. Only numpy is used for permutation entropy, and for sample entropy scipy is used solely as a vectorised max-norm distance primitive, never for the entropy formula itself. This was a deliberate choice so that the reproduction tests the paper's equations, not someone else's implementation of them.
- Every parameter the paper leaves unspecified — sliding-window length, filter type and order, detrending algorithm, ordinal-pattern tie-breaking rule — is exposed as a named, documented, swappable parameter and logged in docs/paper_parameters.md with a FACT / ASSUMPTION / NOT SPECIFIED tag, so it is always clear which numbers came from the paper and which were filled in by the reproducer.

3. Methodology Implemented

3.1 Permutation Entropy (H)

Permutation entropy quantifies the temporal disorder of a signal by looking at the relative ordering (the “ordinal pattern”) of consecutive samples within a sliding embedding window, rather than their raw amplitude. It is fast to compute and insensitive to noise amplitude, which is why the original authors chose it as a whistle detector. Following the paper, the implementation uses an embedding dimension $m = 6$ and a time delay $\tau = 1$. Values range from 0 (fully ordered / low biological activity) to 1 (maximal disorder / high biophonic complexity).

3.2 Sample Entropy (SE)

Sample entropy measures the (negative log) conditional probability that two sequences of length d that are similar within a tolerance r remain similar at length $d + 1$, under a max-norm distance. It is more robust to short or noisy signals than related measures such as approximate entropy and was used in the paper primarily

as a click detector. Following the paper, the implementation uses embedding dimension $d = 2$ and tolerance $r = 0.5 \times$ the signal's standard deviation.

3.3 Parameter Ledger

Because the paper does not specify every implementation detail, the repository maintains an explicit parameter ledger rather than guessing silently. The table below summarises its structure.

Parameter	Status	Value used
Embedding dimension m (permutation entropy)	FACT (stated in paper)	6
Time delay τ (permutation entropy)	FACT (stated in paper)	1
Embedding dimension d (sample entropy)	FACT (stated in paper)	2
Tolerance r (sample entropy)	FACT (stated in paper)	$0.5 \times$ SD of the signal
Sliding-window length	NOT SPECIFIED $\rightarrow$ documented assumption	Exposed as a named, swappable parameter
Filter type / order (pre-processing)	NOT SPECIFIED $\rightarrow$ documented assumption	Exposed as a named, swappable parameter
Detrending algorithm	NOT SPECIFIED $\rightarrow$ documented assumption	Exposed as a named, swappable parameter
Ordinal-pattern tie-breaking rule	NOT SPECIFIED $\rightarrow$ documented assumption	Exposed as a named, swappable parameter

Because these four items are not pinned down by the paper, any accuracy or correlation figure the pipeline eventually produces will be sensitive to them. The repository's approach — expose, name, and document each one rather than hard-coding a silent guess — is the right way to keep the reproduction auditable, but it also means a like-for-like match to the paper's exact numbers can never be fully guaranteed even with the original data.

4. Pipeline Walkthrough (Notebooks)

The pipeline is organised as nine ordered scripts (mirrored as notebooks) that are meant to be run in sequence. Each stage's status, as actually executed during the internship, is summarised below.

#	Script	Stage	Status
02	audio_inspection.py	Load and visually/statistically inspect the 8 Mendeley WAV files	Runs successfully
03	permutation_entropy.py	Compute H on the sample clips	Runs successfully
04	sample_entropy.py	Compute SE on the sample clips	Runs successfully
05	preprocessing.py	Apply filtering / detrending prior to entropy estimation	Runs successfully
06	short_clip_reproduction.py	End-to-end run on the 8 short validation clips	Runs successfully
07	long_duration_pipeline_BLOCKED.py	Reproduce the on/off detection dynamic (paper Figs. 1, 2, 6, 7)	Blocked — built and unit-tested on synthetic placeholder data

#	Script	Stage	Status
			only; no long-duration field recording is publicly available
08	confusion_matrix_BLOCKED.py	Reproduce Table 1's confusion matrices and accuracy figures	Blocked — built and unit-tested on synthetic placeholder data only; no manual annotations are publicly available
09	computational_benchmark.py	Benchmark runtime / computational cost of H and SE	Runs successfully
10	final_report_generation.py	Auto-generate the results tables and figures into results/	Runs successfully

Notebooks 07 and 08 are deliberately named with a “_BLOCKED” suffix in the repository itself. When run, they do not print or fabricate any numbers resembling the paper's — they raise an explicit message explaining that the required data is unavailable. This is a notable and appropriate design decision: it prevents a reader from mistaking a placeholder run for a genuine reproduction.

5. Dataset Used

The only publicly released data associated with the paper is a set of eight short WAV clips on Mendeley Data (DOI 10.17632/zv6ygrptr4.1), intended by the original authors as illustrative validation examples rather than as the long-duration recordings behind the paper's main results. Their contents are:

#	File	Duration (s)	Description
1	Whistle1.wav	3	3 whistles of varying intensity with overlapping snaps and clicks
2	Whistle2.wav	3.4	4 whistles with overlapping snaps and clicks
3	Whistle3.wav	5	5 whistles of different types with clicking
4	Whistle4.wav	5	No whistles (negative control)
5	Click1.wav	2	Click train of varying intensity ($t = 0.011$)
6	Click2.wav	3	Clicks with overlapping whistles ($t = 0.011$)
7	Click3.wav	1.5	Varying-intensity clicks in the train ($t = 0.011$)
8	Click4.wav	0.5	Click train (can be tested by varying t between 0.001–0.011)

Because every clip is short and continuously vocalising, none of them contain the alternating noise/vocalisation periods that the paper's on/off detection accuracy figures are built on. This is precisely why notebooks 07 and 08 are blocked: the input data they would need simply is not this dataset.

6. Results: What Could and Could Not Be Reproduced

6.1 Successfully reproduced

- Both entropy algorithms (H and SE) implemented from the paper's equations and validated against the 8 short Mendeley clips.
- Preprocessing pipeline (filtering / detrending) built and connected to the entropy stage.
- A full pytest suite covering the algorithm implementations and pipeline stages.
- A computational-cost benchmark of both metrics.
- A parameter ledger distinguishing facts stated in the paper from assumptions the reproducer had to make.
- An automatically generated results bundle (figures/, tables/, metrics/) from the parts of the pipeline that can run on public data.

6.2 Not reproducible with public data

Paper result	Reported value	Reproducible?	Blocking reason
Table 1 — whistle detection accuracy	96.6%	No	Requires manual on/off annotations on long-duration recordings, not publicly released
Table 1 — click detection accuracy	97.8%	No	Same as above
Correlation, H vs. annotated activity	Pearson $r > 0.95$	No	Requires the same long-duration annotated recordings
Correlation, scenario 2	Pearson $r = 0.89$	No	Requires the same long-duration annotated recordings
Correlation, scenario 3	Pearson $r = 0.97$	No	Requires the same long-duration annotated recordings
Figs. 1, 2, 6, 7 — on/off detection dynamic	—	No	Needs recordings with alternating noise/vocalisation periods; the 8 Mendeley clips are all continuously vocalising

The original long-duration ETS/HB recordings and the manual annotations behind Table 1 and the correlation figures were never deposited alongside the paper. They exist only with the corresponding authors. This is a data-availability gap in the original publication, not a flaw in the reproduction code.

7. Author Correspondence

Recognising that the missing data is the single blocker standing between this repository and a full reproduction, the project prepared a draft outreach message to the paper's corresponding authors requesting access to the original long-duration recordings and their manual annotations (docs/author_correspondence.md). As of this report, that request had not yet been sent/answered within the repository's own records — it is documented as the clear next step rather than as a completed action.

8. Engineering and Documentation Quality

A few choices in this repository are worth highlighting as good practice for future reproduction work, beyond the specific dolphin-detection result:

- Implementing both entropy formulas from first principles (numpy only) rather than importing an existing entropy package, so the test is of the paper's method, not of someone else's code.
- Naming every unspecified parameter explicitly instead of silently hard-coding a guess, with a FACT / ASSUMPTION / NOT SPECIFIED tag per parameter.
- Refusing to fabricate placeholder numbers for the blocked confusion-matrix and correlation results — the blocked notebooks raise an explicit message instead of printing anything that could be mistaken for a real result.
- A dedicated, continuously updated status document (reproduction_audit.md) that states plainly what has and has not been verified, rather than leaving that determination to the reader.
- A pytest suite covering the algorithm layer, so future changes to preprocessing or parameters can be checked against known behaviour.

9. Conclusions and Recommendations

This internship succeeded in building a faithful, independently-implemented, and well-tested version of the two entropy detectors at the heart of Siddagangaiah et al. (2020). It did not succeed in reproducing the paper's headline accuracy and correlation numbers, because those numbers depend on data that the original authors have not made public. That distinction is the central, honest conclusion of the work: the algorithm is reproducible; the paper's specific reported performance, on present public data, is not independently verifiable.

Recommended next steps:

- Send the drafted request (docs/author_correspondence.md) to the corresponding authors for the long-duration ETS/HB recordings and manual annotations.
- If and when that data arrives, run notebooks 07 and 08 for real and compare the resulting confusion matrices and Pearson correlations directly against Table 1 and Figures 1, 2, 6 and 7 of the paper.
- Run a small sensitivity analysis over the NOT SPECIFIED parameters (window length, filter choice, detrending, tie-breaking) to understand how much they could move the final accuracy figures, since the paper does not pin these down.
- Consider reaching out to other groups that have cited or built on this paper (e.g., convolutional/transfer-learning follow-ups) to see whether any of them obtained the original long-duration data under separate agreement.

References

Siddagangaiah, S., Chen, C.-F., Hu, W.-C., Akamatsu, T., McElligott, M., Lammers, M.O., Pieretti, N. (2020). Automatic detection of dolphin whistles and clicks based on entropy approach. *Ecological Indicators*, 117, 106559. <https://doi.org/10.1016/j.ecolind.2020.106559> (open access, CC BY 4.0).

Mendeley Data. Data for: Automatic detection of dolphin whistles and clicks based on entropy approach. DOI: 10.17632/zv6ygrpr4.1.

Code repository: github.com/nikhatraghna/Automatic-detection-of-dolphin-whistles-and-clicks-based-on-entropy-approach.